Speed Tracker Module

Technical plan: what to have, how to build it, and the complete backend

Draft v1.0 | 16 October 2026

What we are building. A module inside your existing authority website that turns GPS data from vehicles into live speed maps per road segment, flags problems automatically, and connects them to complaints (potholes, broken roads).

What you get. (1) Overspeed hotspots, (2) slowdown / hard-braking spots that often mean road damage, (3) proof that a repair actually helped, (4) complaints ranked by how many vehicles pass over them.

Biggest risk (read section 12 first). The code is the easy part. With no vehicles sending data, the map stays empty. You need a plan to get GPS data from at least a few dozen vehicles in your pilot area.

1. Scope and interpretation

In this plan, **speed tracker** means: collect real vehicle speeds on the roads of one district, attach them to road segments, and show them in the existing dashboard next to the complaints. Why authorities care:

- **Overspeed hotspots:** segments where the 85th-percentile speed is above the limit (places for speed breakers, signage, enforcement).
- **Road-damage evidence:** a sudden slowdown or a cluster of hard braking at one spot usually means a pothole or broken patch. This supports existing complaints and also finds unreported ones.
- **Repair verification:** after a complaint is marked resolved, did speeds on that segment recover?
- **Prioritisation:** rank open complaints by traffic exposure (how many vehicles really pass there).

Be careful with the claim. Phone-GPS speed is good for statistics (medians and percentiles over many vehicles). It is **not** a certified speed-measuring device, so never present one reading as proof of a violation, and do not build challans on it. Out of scope: number-plate recognition, fines, tracking individual people.

Alternative meaning. If by speed tracker you meant how fast complaints get resolved (time-to-fix, backlog age), that needs no new data source. It is covered as Module B in section 8 and is a cheap quick win you can ship in a day or two.

Assumptions (correct me if any is wrong)

#	Assumption	If wrong
1	Frontend is React (Vite or CRA) with a map library already used for complaint pins (Leaflet or MapLibre).	Components in section 6 stay the same idea; only imports change.
2	Each complaint already stores latitude and longitude.	Add a location picker or geocode the address before section 7 works.
3	Backend does not exist yet, or is small enough to sit next to the new FastAPI service.	If you already have Node/Express, reuse its auth and call the new service for speed endpoints.
4	Pilot area is one district / city, a few hundred road segments to a few thousand.	Beyond ~5,000 segments in one view, switch to vector tiles (section 6).

2. What you need (checklist)

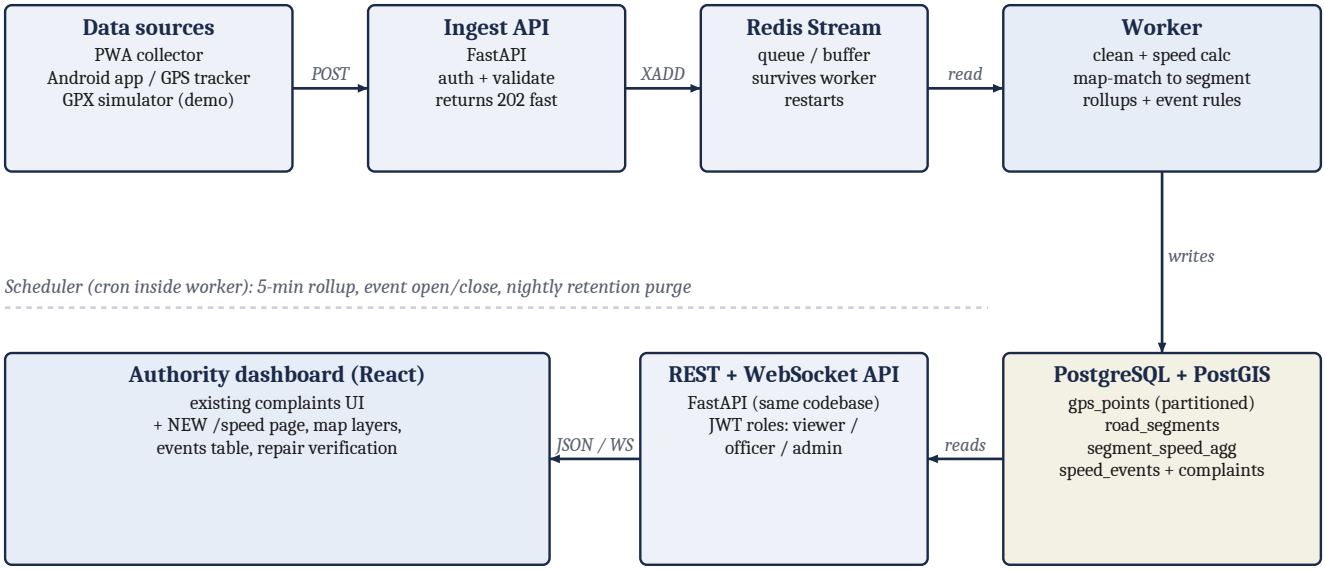
Area	Item	Why / notes
Software	Python 3.11+, Node 20+, PostgreSQL 16 with PostGIS 3, Redis 7, Docker + Docker Compose, Git	All free. Docker keeps the setup identical on your laptop and the server.
Map data	OpenStreetMap extract for your district (download India from Geofabrik, clip with `osmium extract -b`)	Gives you the road network. Import with `osm2pgsql` into PostGIS.
Speed limits	A table of default limits per road class, editable per segment by an admin	OSM `maxspeed` is often missing in India. Use limits from your authority's notification, not guesses.
Test data	3 to 5 GPX recordings of real drives (any GPS logger app) plus the simulator in the appendix	Lets you develop and demo without waiting for real users.
Data source	Collector client: PWA first, native Android later, or dedicated GPS trackers on official vehicles	See the comparison table below. This decides your real data quality.
Map tiles	A tile provider with a free tier, or your own tile server	The public OSM tile server has a usage policy and is not meant for production traffic.
Hosting	One small VPS (2 vCPU, 4 GB RAM) is enough for the pilot	Needs HTTPS: browsers only allow geolocation on secure origins.
People / permission	Written approval from the authority for the pilot area and a short consent text for contributors	Needed before collecting anyone's location. See section 9.

Choosing the data collector

Option	Effort	Pros	Cons
A. PWA in the browser	Low (1 to 2 days)	No app-store, quick demo, reuse React skills	Tracking mostly stops when the screen locks or the tab goes to background. Good for demo only.
B. Native Android (React Native / Flutter)	Medium (1 to 2 weeks)	Real background tracking, better battery control	Needs a foreground-service notification, permissions flow, Play Store or APK distribution.
C. GPS trackers on official vehicles (buses, garbage trucks, jeeps)	Low software, some hardware cost	Reliable, continuous, legally clean (your own vehicles)	Hardware cost and SIM data. Coverage limited to routes those vehicles run.

Recommendation: build option A plus the simulator for the pilot demo, and plan option C for real coverage. Treat B as phase two.

3. System architecture



Data flow in eight steps

1. The collector sends a batch of GPS fixes every ~10 seconds (max 500 fixes per batch) with the device's API key.
2. The Ingest API authenticates the key, validates the payload, pushes it to a Redis stream and returns `202`. It does no heavy work.
3. The worker reads the stream and cleans each trip: drops bad fixes, computes speed from consecutive points, smooths it.
4. Each fix is matched to a road segment (nearest segment with a heading check; Valhalla map-matching as an upgrade).
5. Cleaned points are stored in the partitioned `gps_points` table.
6. Every 5 minutes a rollup writes median, P85, min, max, trip count per segment into `segment_speed_agg`.
7. The detector applies the rules in section 5.6, opens or closes rows in `speed_events` and pushes them to the WebSocket.
8. The dashboard reads segments, events and complaints through the REST API, and live updates through the WebSocket.

> **Why a queue?** Ingest must stay fast and must not lose data when the worker restarts or slows down. **Why one codebase?** `api` and `worker` are two processes of the same Python project. Separate microservices are not needed at this scale.

4. Data model

All tables live in the same PostgreSQL database as your complaints, so a complaint and the traffic around it can be joined in one query. Column names for `complaints` are examples; adapt them to your schema.

Listing 4.1 - devices, trips, gps_points

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE devices (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  device_hash  TEXT UNIQUE NOT NULL,      -- salted hash, never the raw phone id
  api_key_hash TEXT NOT NULL,
  platform     TEXT NOT NULL CHECK (platform IN ('pwa','android','ios','tracker','simulator')),
  consent_at   TIMESTAMPTZ NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  revoked_at   TIMESTAMPTZ
);

CREATE TABLE trips (
  id            UUID PRIMARY KEY,         -- generated by the client
  device_id     UUID NOT NULL REFERENCES devices(id),
  started_at    TIMESTAMPTZ NOT NULL,
  ended_at      TIMESTAMPTZ,
  vehicle_class TEXT NOT NULL DEFAULT 'unknown'   -- car | bike | bus | truck | unknown
);

CREATE TABLE gps_points (
  id            BIGINT GENERATED ALWAYS AS IDENTITY,
  trip_id       UUID NOT NULL,
  recorded_at   TIMESTAMPTZ NOT NULL,
  geom          GEOMETRY(Point, 4326) NOT NULL,
  speed_kmh     REAL,                     -- cleaned and smoothed
  raw_speed_kmh REAL,                     -- as reported by the device, if any
  accuracy_m    REAL,
  heading       REAL,
  segment_id    BIGINT,                   -- filled by map matching
  PRIMARY KEY (id, recorded_at)
) PARTITION BY RANGE (recorded_at);

CREATE TABLE gps_points_2026_10_16 PARTITION OF gps_points
  FOR VALUES FROM ('2026-10-16') TO ('2026-10-17');   -- create daily (pg_partman or a job)
CREATE INDEX ON gps_points (segment_id, recorded_at);
CREATE INDEX ON gps_points USING BRIN (recorded_at);
```

Listing 4.2 - segments, aggregates, events, complaint changes

```
CREATE TABLE road_segments (
  id              BIGSERIAL PRIMARY KEY,
  osm_way_id      BIGINT,
  name            TEXT,
  highway         TEXT,                   -- OSM road class
  speed_limit_kmh SMALLINT,
  limit_source    TEXT NOT NULL DEFAULT 'default',   -- osm | default | manual
  length_m        REAL,
  geom            GEOMETRY(LineString, 4326) NOT NULL
);
CREATE INDEX ON road_segments USING GIST (geom);

CREATE TABLE segment_speed_agg (
  segment_id   BIGINT NOT NULL REFERENCES road_segments(id),
  bucket_start TIMESTAMPTZ NOT NULL,      -- 5-minute buckets
  n_points     INT NOT NULL,
  n_trips      SMALLINT NOT NULL,
  avg_kmh REAL, p50_kmh REAL, p85_kmh REAL, min_kmh REAL, max_kmh REAL,
  PRIMARY KEY (segment_id, bucket_start)
);

CREATE TABLE speed_events (
  id           BIGSERIAL PRIMARY KEY,
  type         TEXT NOT NULL CHECK (type IN ('OVERSPEED','SLOWDOWN','DECEL_CLUSTER')),
  segment_id   BIGINT NOT NULL REFERENCES road_segments(id),
  geom         GEOMETRY(Point, 4326),
  first_seen   TIMESTAMPTZ NOT NULL,
  last_seen    TIMESTAMPTZ NOT NULL,
  severity     SMALLINT NOT NULL CHECK (severity BETWEEN 1 AND 3),
  metric       JSONB NOT NULL,            -- e.g. {"p85":71,"limit":40,"n_trips":9}
  status       TEXT NOT NULL DEFAULT 'open'
               CHECK (status IN ('open','acknowledged','linked','closed')),
  complaint_id BIGINT REFERENCES complaints(id)
);

-- changes to your EXISTING complaints table (adapt column names)
ALTER TABLE complaints ADD COLUMN geom       GEOMETRY(Point, 4326);
ALTER TABLE complaints ADD COLUMN segment_id BIGINT REFERENCES road_segments(id);
UPDATE complaints SET geom = ST_SetSRID(ST_MakePoint(lng, lat), 4326) WHERE geom IS NULL;
CREATE INDEX ON complaints USING GIST (geom);
```

- **Retention:** raw `gps_points` for 30 days (drop old partitions), `segment_speed_agg` for 1 to 2 years. Aggregates carry no personal data.
- **Segments:** import OSM ways, split at junctions and then into pieces of about 150 m, so one colour on the map means one short stretch of road.

5. Backend design

5.1 Stack

Layer	Choice	Reason
API + worker	Python 3.11, FastAPI, Pydantic v2, SQLAlchemy 2 + Alembic	Fast to write, strong typing, and Python suits the data processing.
Database	PostgreSQL 16 + PostGIS	Spatial joins, distance, nearest-segment, partitioning in one system.
Queue / cache	Redis 7 (Streams)	Simple, enough for hundreds of requests per second.
Realtime	WebSocket (or Server-Sent Events)	Push new events to open dashboards.
Map matching	PostGIS nearest-segment (MVP), Valhalla `trace_attributes` (upgrade)	Valhalla returns the OSM way id per matched edge, but is one more service to run.
Auth	Reuse your existing officer login (JWT); device API keys for collectors	No second login system.

5.2 Project structure

```
backend/
  app/
    main.py              # FastAPI app, routers, CORS, startup
    config.py            # settings from environment variables
    db.py                # async engine / session
    api/
      ingest.py          # POST /v1/ingest/batch
      devices.py         # register / revoke device
      segments.py        # segment map + stats
      events.py          # list / acknowledge / close / create complaint
      complaints.py      # speed-context + repair verification
      settings.py        # thresholds and speed limits (admin)
      stream.py          # WebSocket
    core/ security.py  rate_limit.py
    pipeline/
      clean.py           # filters + speed calculation
      match.py           # map matching
      aggregate.py       # 5-minute rollups
      detect.py          # event rules (hysteresis)
      verify.py          # repair verification
    worker.py            # stream consumer + scheduler
  migrations/            # Alembic
  scripts/ import_osm.sh  seed_limits.sql  simulate_gpx.py
  tests/
```

5.3 API

Method and path	Who	Purpose
`POST /v1/devices/register`	Public, rate-limited, needs consent flag	Create a device, return its API key once.
`POST /v1/ingest/batch`	Device key	Send a batch of GPS fixes. Returns 202.
`GET /v1/segments?bbox=&at=`	Officer	GeoJSON of segments in view with latest speed stats.
`GET /v1/segments/{id}/stats?from=&to=`	Officer	Time series (5-minute buckets) for the chart.
`GET /v1/events?type=&status=&from=&to=`	Officer	List events with filters.
`PATCH /v1/events/{id}`	Officer	Acknowledge or close an event.
`POST /v1/events/{id}/complaint`	Officer	Create a complaint from an event and link both.
`GET /v1/complaints/{id}/speed-context`	Officer	Speed stats near a complaint and repair verification result.
`GET/PUT /v1/settings/thresholds`	Admin	Read or change rule thresholds.
`PUT /v1/segments/{id}/limit`	Admin	Set a manual speed limit for a segment.
`WS /v1/stream/events`	Officer	Live event feed.
`GET /healthz`	Public	Health check for deployment.

5.4 Ingest contract

Listing 5.1 - request and response (spd is m/s as the browser reports it, may be null)

```
POST /v1/ingest/batch          X-API-Key: <device key>
{
  "trip_id": "6f1c2d1e-2c61-4c55-9c4b-0b2a6a1f9a11",
  "vehicle_class": "car",
  "fixes": [
    { "t": "2026-10-16T09:41:03Z", "lat": 31.7087, "lon": 76.9320,
      "acc": 6.0, "spd": 9.8, "hdg": 112.0 },
    { "t": "2026-10-16T09:41:04Z", "lat": 31.7087, "lon": 76.9322,
      "acc": 5.5, "spd": null, "hdg": null }
  ]
}
-> 202 { "accepted": 2 }
```

Listing 5.2 - ingest endpoint

```
# app/api/ingest.py
from datetime import datetime
from fastapi import APIRouter, Depends, Header
from pydantic import BaseModel, Field

router = APIRouter(prefix="/v1/ingest")

class Fix(BaseModel):
    t: datetime
    lat: float = Field(ge=-90, le=90)
    lon: float = Field(ge=-180, le=180)
    acc: float | None = Field(default=None, ge=0, le=500)
    spd: float | None = Field(default=None, ge=0, le=100)     # m/s, optional
    hdg: float | None = Field(default=None, ge=0, le=360)

class Batch(BaseModel):
    trip_id: str
    vehicle_class: str = "unknown"
    fixes: list[Fix] = Field(min_length=1, max_length=500)

@router.post("/batch", status_code=202)
async def ingest(batch: Batch, x_api_key: str = Header(...), redis=Depends(get_redis)):
    device = await authenticate_device(x_api_key)       # 401 if unknown or revoked
    await redis.xadd("gps:in", {"device": str(device.id), "payload": batch.model_dump_json()})
    return {"accepted": len(batch.fixes)}
```

Validation rules: reject timestamps more than 5 minutes in the future or more than 24 hours old (allow old ones only when `ALLOW_OLD_TIMESTAMPS=true`, for the GPX simulator), cap batch size at 500, rate-limit per device key (for example 12 requests per minute), and never trust `spd` blindly. It is only a cross-check.

5.5 Processing pipeline

Step	Rule	Default
Poor accuracy	Drop a fix if reported accuracy is worse than this	30 m
Out of order / duplicate	Drop if its time is not after the previous fix	n/a
Gaps	Start a new sub-trip if the gap between fixes is longer than this	30 s
Teleport	Drop if computed speed is above this	160 km/h
Stationary jitter	Set speed to 0 if computed speed is below this	2 km/h
Smoothing	Median of 3 consecutive speeds	window 3
Dwell points	Ignore points below 3 km/h for more than 60 s (parking, traffic signal)	3 km/h, 60 s
Trip edges	Trim the first and last 200 m of every trip before storing (privacy)	200 m

Listing 5.3 - cleaning and speed calculation (pipeline/clean.py)

```
from math import radians, sin, cos, asin, sqrt

def haversine_m(lat1, lon1, lat2, lon2):
    R = 6371000.0
    dlat, dlon = radians(lat2 - lat1), radians(lon2 - lon1)
    a = sin(dlat / 2) ** 2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon / 2) ** 2
    return 2 * R * asin(sqrt(a))

def clean_and_speed(points, max_acc_m=30, max_kmh=160, max_gap_s=30):
    """points: time-sorted dicts {t (epoch s), lat, lon, acc}. Returns points with 'kmh'."""
    out, prev = [], None
    for p in points:
        if p["acc"] is not None and p["acc"] > max_acc_m:
            continue                                # poor fix
        if prev is not None:
            dt = p["t"] - prev["t"]
            if dt <= 0 or dt > max_gap_s:           # duplicate, out of order, or gap
                prev = p
                continue
            v = haversine_m(prev["lat"], prev["lon"], p["lat"], p["lon"]) / dt * 3.6
            if v > max_kmh:                         # GPS jump, keep old prev
                continue
            out.append({**p, "kmh": 0.0 if v < 2 else v})
        prev = p
    return out
```

Listing 5.4 - nearest-segment candidates

```
-- MVP map matching: nearest segment within ~30 m (0.0003 degrees is roughly 33 m)
SELECT s.id
FROM road_segments s
WHERE ST_DWithin(s.geom, ST_SetSRID(ST_MakePoint(:lon, :lat), 4326), 0.0003)
ORDER BY s.geom <-> ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)
LIMIT 5;      -- then pick the candidate whose bearing is closest to the fix heading
```

Nearest-segment matching is wrong at junctions and on parallel roads (a flyover above a road, a service lane next to a highway). The heading check reduces this. If wrong matches show up in testing, move to Valhalla `trace_attributes`, which matches the whole trace and returns the way id of each edge.

Listing 5.5 - 5-minute rollup (run every 5 minutes; re-covers the last 30 minutes for late data)

```
INSERT INTO segment_speed_agg
  (segment_id, bucket_start, n_points, n_trips, avg_kmh, p50_kmh, p85_kmh, min_kmh, max_kmh)
SELECT segment_id,
       date_bin('5 minutes', recorded_at, TIMESTAMPTZ '2026-01-01 00:00+00'),
       count(*), count(DISTINCT trip_id), avg(speed_kmh),
       percentile_cont(0.50) WITHIN GROUP (ORDER BY speed_kmh),
       percentile_cont(0.85) WITHIN GROUP (ORDER BY speed_kmh),
       min(speed_kmh), max(speed_kmh)
FROM gps_points
WHERE segment_id IS NOT NULL AND speed_kmh IS NOT NULL
  AND recorded_at >= now() - interval '30 minutes'
GROUP BY 1, 2
ON CONFLICT (segment_id, bucket_start) DO UPDATE SET
  n_points = EXCLUDED.n_points, n_trips = EXCLUDED.n_trips, avg_kmh = EXCLUDED.avg_kmh,
  p50_kmh = EXCLUDED.p50_kmh, p85_kmh = EXCLUDED.p85_kmh,
  min_kmh = EXCLUDED.min_kmh, max_kmh = EXCLUDED.max_kmh;
```

5.6 Event detection rules

Event	Condition (per segment)	Severity	Why
OVERSPEED	P85 above limit x 1.10 in 3 consecutive buckets, with at least 5 trips	1: ratio 1.10 to 1.25, 2: 1.25 to 1.5, 3: above 1.5	P85 is the standard traffic-engineering measure, robust to a few fast outliers.
SLOWDOWN	P50 below 0.6 x the segment's own baseline for the same hour of the week (median of last 4 weeks) in 3 buckets, at least 5 trips. Until 2 weeks of history exist, use P50 below 0.5 x limit.	1: ratio 0.5 to 0.6, 2: 0.35 to 0.5, 3: below 0.35	Comparing a segment with itself avoids flagging naturally slow roads (market street, hill hairpin).
DECEL_CLUSTER	In a 30 m cell, at least 4 different trips lose more than 50 % of their speed within 3 s, in the last 24 h	By number of trips: 4-7 = 1, 8-15 = 2, 16+ = 3	Points to one spot: pothole, unmarked speed breaker, bad patch.

- **Hysteresis:** open an event after the condition holds for 3 buckets (15 min), close it after it is false for 6 buckets (30 min). This stops events flickering.
- **Minimum evidence:** never raise or colour a segment from fewer than 5 trips. Show it grey as **no data**. No data does not mean free-flowing.
- **All thresholds live in a settings table** and the admin screen, not in code, because you will tune them on real data.

5.7 Realtime

When the detector opens or changes an event it publishes JSON to a Redis channel. The API's WebSocket handler subscribes and forwards it to every connected officer. The dashboard also refetches segments every 30 seconds, so a dropped WebSocket does not leave the map stale.

5.8 Security

- HTTPS everywhere. Officers use the existing JWT with roles `viewer`, `officer`, `admin`. Check the role in every endpoint (not only in the UI).
- Device keys are random 32-byte values, shown once, stored only as a hash. Support revoke, and rate-limit per key and per IP.
- Use parameterised queries only (SQLAlchemy bind parameters). Validate all input with Pydantic. Restrict CORS to your dashboard origin.
- Write an audit log row for every acknowledge, close, threshold change and manual speed-limit change (who, what, when).

6. Frontend: adding the Speed page to your existing site



Figure 6.1 - Layout of the new `/speed` page. The left filters, map and right drawer reuse your existing page shell; the bottom strip is the events table.

Components

File (src/features/speed/)	What it does
`SpeedPage.tsx`	Route `/speed`. Holds filter state and lays out map, drawer and events table.
`SpeedMap.tsx`	Uses the same map library as your complaints map. Adds the speed layer and the complaint pins as a second layer.
`SegmentLayer.tsx`	Draws segments from GeoJSON, coloured by the ratio median speed / limit (scale below). Click selects a segment.
`SegmentDrawer.tsx` + `SpeedChart.tsx`	Details of the selected segment: limit, median, P85, trips, 24 h chart (Recharts), linked complaints.
`EventsTable.tsx`	List of events with filters. Actions: acknowledge, close, create complaint.
`RepairVerificationCard.tsx`	Shown on a resolved complaint: speeds before and after, verdict (section 7).
`SettingsPanel.tsx` (admin)	Edit thresholds and per-segment speed limits.
`api.ts`, `hooks.ts`, `types.ts`	Typed API client, React Query hooks, shared types.

Colour	Meaning (median speed / limit)
Purple	Above 1.1: driving over the limit
Green	0.8 to 1.1: near the limit, normal flow
Amber	0.5 to 0.8: slow
Red	Below 0.5: very slow, possible blockage or damage
Grey	Fewer than 5 trips in the window: no data

Listing 6.1 - data hook (Vite assumed; with CRA use process.env.REACT_APP_API_URL)

```
// src/features/speed/hooks.ts
import { useQuery } from "@tanstack/react-query";

export function useSegments(bbox: string, atISO?: string) {
  return useQuery({
    queryKey: ["segments", bbox, atISO],
    queryFn: async () => {
      const url = `${import.meta.env.VITE_API_URL}/v1/segments?bbox=${bbox}` +
                  (atISO ? `&at=${atISO}` : "");
      const r = await fetch(url, { headers: { Authorization: `Bearer ${getToken()}` } });
      if (!r.ok) throw new Error(`segments ${r.status}`);
      return (await r.json()) as GeoJSON.FeatureCollection;
    },
    refetchInterval: 30_000,
    staleTime: 15_000,
  });
}
```

Integration steps

- Add `VITE_API_URL` and the map settings to your env file.
- Add the route `/speed` and a nav item, guarded by the same auth wrapper as your other pages.
- Install `@tanstack/react-query` and `recharts` if not already present.
- Create the `speed` feature folder and build in this order: hooks, SegmentLayer, SegmentDrawer, EventsTable, filters, WebSocket updates.
- Add a small "Traffic" tab to your existing complaint detail view that renders `RepairVerificationCard` and the speed stats for that complaint.
- Performance: up to roughly 5,000 segments in view, GeoJSON is fine. Beyond that, serve vector tiles (Martin or pg_tileserv) and use MapLibre.

7. Linking speed data with complaints

This is what makes the module more than a second map. Three features, all based on one idea: every complaint is snapped to a road segment.

Listing 7.1 - snap complaints to segments (run on insert and once for old rows)

```
UPDATE complaints c
SET segment_id = (
  SELECT s.id FROM road_segments s
  WHERE ST_DWithin(s.geom::geography, c.geom::geography, 30)    -- 30 m
  ORDER BY s.geom <-> c.geom
  LIMIT 1)
WHERE c.segment_id IS NULL AND c.geom IS NOT NULL;
-- for large tables use a degree-based pre-filter or a geography index to keep the GIST index useful
```

- **A. Traffic context:** on each complaint, show how many vehicles pass per day and the speed profile around it. Sort open complaints by exposure (`n_trips` per day) so the busiest potholes surface first.
- **B. Suggest complaints:** when a SLOWDOWN or DECEL_CLUSTER event has no complaint within 50 m, show "Possible unreported road damage" with a one-click **Create complaint** (it pre-fills location and evidence).
- **C. Repair verification:** when a complaint becomes `resolved` at time R, compare the segment's median speed in the 14 days before R with the 14 days after R+1 day. Verdict:

Verdict	Rule (starting values, tune them)
Verified improved	Median speed up by at least 15 % and at least 30 trips in each window
No change detected	Change under 15 % with enough trips (suggests the repair did not matter, or was elsewhere)

Verdict	Rule (starting values, tune them)
Not enough data	Fewer than 30 trips in either window. Say so; do not guess.

> **Honest limit:** speed recovery is evidence, not proof. Weather, festivals, other road works and school timing also change speeds. Show the verdict with the two numbers behind it, and compare same weekdays and hours where possible.

8. Module B: complaint resolution speed (optional, quick win)

If your authority also wants to see how fast complaints are handled, you can ship this from the existing data alone: time to first response, time to resolve, age of open complaints, per category and per officer or area. Adapt column names.

Listing 8.1 - median time to fix by category

```
SELECT category,
       count(*)                                                          AS resolved,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY resolved_at - created_at) AS median_time_to_fix
FROM complaints
WHERE status = 'resolved' AND resolved_at >= now() - interval '90 days'
GROUP BY category
ORDER BY median_time_to_fix DESC;
```

UI: three number cards (median fix time, open backlog, oldest open) and a bar chart per category. Be careful presenting per-officer numbers: they invite gaming and unfair comparison. Start with per-category and per-area.

9. Privacy, security and legal

- **Consent and purpose:** a short plain-language screen before the first upload: what is collected (location, time, speed), why (road safety and road quality), who sees it (authority officers, aggregated), how long it is kept. A revoke option must work.
- **Data minimisation:** no names, phone numbers or number plates. Store a salted hash as the device id. Trim the first and last 200 m of every trip so home and workplace are not stored.
- **Aggregate-only dashboard:** officers see segment statistics, never a moving dot for one vehicle. Hide any value computed from fewer than 5 trips.
- **Retention:** raw points deleted after 30 days; aggregates kept longer. Automate it and test it.
- **Law:** collecting location of people in India falls under the Digital Personal Data Protection Act, 2023. Check the current status of its rules and your institute's or authority's data policy before collecting real data. This is not legal advice; ask your authority's legal contact.
- **Enforcement:** keep this module analytical. Using it for fines needs certified equipment and a legal process outside this plan.

10. Testing and validation

Level	What to test	How
Unit	`haversine_m`, cleaning rules, percentile and event rules, hysteresis, repair verdict	pytest with small fixed inputs, including bad data (duplicates, jumps, gaps).
Integration	Ingest to worker to database to API using real PostGIS	Docker Compose test profile; replay a GPX file and assert segment stats.
Accuracy	Compare computed speed with a known reference	Drive one measured route with a car speedometer or a second GPS logger; report mean and worst error.
Map matching	Wrong-segment rate at junctions and parallel roads	Label 50 points by hand, measure the match rate, decide if Valhalla is needed.
Load	Ingest stays fast under many devices	Locust or k6: 200 devices x 1 batch / 10 s = 20 requests/s. Target: p95 under 200 ms, no lost batches.
Frontend	Colours, empty states, role guards	Component tests plus a manual run-through of the demo script.

Capacity check: 200 devices each sending 1 fix per second is 200 rows/s. If they are active 2 hours per day that is about 1.4 million rows per day, and about 43 million in 30 days. That is fine for PostgreSQL with daily partitions and dropping old partitions, and it is the reason retention is not optional.

11. Deployment

Listing 11.1 - local and pilot setup (put Nginx or Caddy with HTTPS in front of api)

```
# docker-compose.yml
services:
  db:
    image: postgis/postgis:16-3.4
    environment: { POSTGRES_DB: roads, POSTGRES_PASSWORD: ${DB_PASSWORD} }
    volumes: [ "pgdata:/var/lib/postgresql/data" ]
  redis:
    image: redis:7-alpine
  api:
    build: ./backend
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000
    env_file: .env
    depends_on: [db, redis]
    ports: [ "8000:8000" ]
  worker:
    build: ./backend
    command: python -m app.worker
    env_file: .env
    depends_on: [db, redis]
volumes: { pgdata: {} }
```

Variable	Example / default	Purpose
`DATABASE_URL`	postgresql+asyncpg://user:pass@db/roads	Database connection
`REDIS_URL`	redis://redis:6379/0	Queue and pub/sub
`JWT_SECRET`	(long random string)	Same secret or public key as your existing login
`DEVICE_KEY_SALT`	(long random string)	Hashing device ids and keys
`ALLOW_OLD_TIMESTAMPS`	false (true only in dev)	Lets the GPX simulator send past times
`RETENTION_RAW_DAYS`	30	Raw GPS retention
`MIN_TRIPS_FOR_STATS`	5	Minimum evidence for colouring or events

Variable	Example / default	Purpose
`CORS_ORIGINS`	https://your-dashboard.example	Allowed browser origin

- **Backups:** nightly `pg_dump` of the database to a different machine; test one restore before the demo.
- **CI:** run lint, unit tests and the integration test on every push (GitHub Actions). Block merges when tests fail.
- **Monitoring:** log queue length, ingest rate, worker lag and errors. A stuck worker is the most likely silent failure.

12. Roadmap and risks

Phase	Work	Done when	Days
0. Setup	Repo, Docker Compose, PostGIS, OSM import, speed-limit defaults, GPX test drives	Roads of the pilot area visible in the database	1-2
1. Ingest	Device registration, ingest endpoint, Redis stream, simulator	A replayed GPX trip appears as raw points	2-3
2. Pipeline	Cleaning, speed calc, matching, rollup, tests	Segment stats match a hand-checked drive	4-5
3. Read API	Segments, stats, events, auth and roles	Postman collection passes	2-3
4. Frontend	Speed page, map layer, drawer, chart, filters, events table	Officer can find a slow segment and open its details	4-5
5. Events + links	Detection rules, hysteresis, complaint snapping, suggest and verify	Event creates a complaint; a resolved complaint shows a verdict	4-5
6. Hardening	Privacy features, retention job, load test, accuracy test, docs	Every item in sections 9 and 10 is implemented and tested	3-4

Total about **20 to 27 working days** for one person who already knows React and is learning PostGIS along the way. These are rough estimates; the pipeline phase is the one most likely to run over.

Risks and honest critique

Risk	Impact	What to do
Cold start: no data. Few vehicles send GPS, so most segments stay grey.	Dashboard looks empty; officers lose trust	Start with GPS trackers or the app on municipal buses, garbage trucks, taxi unions, campus shuttles. Define a pilot target (for example 30 vehicles) before building the UI polish.
GPS error in valleys, tunnels, dense areas	Wrong speeds, wrong segments	Accuracy filter, minimum trips, median and P85 instead of single values.
Missing or wrong speed limits	False OVERSPEED events	Use official limits, mark `limit_source`, show "limit unknown" instead of a default guess.
PWA stops tracking in the background	Short, broken trips	Use PWA for demo only; plan native Android or trackers.
Scope creep (live vehicle tracking, fines)	Privacy and legal exposure, delays	Stay with aggregate road statistics. Write the non-goals into the project README.
Thresholds tuned on no real data	Too many or too few events	Ship thresholds as settings; review events weekly during the pilot.
Map tile and hosting limits	Map breaks under load	Use a proper tile provider or own tiles for anything beyond the demo.

A. Appendix: collector and simulator

Listing A.1 - browser collector

```
// collector.js - browser collector (works only while the page is open and the screen is on)
const API = "https://api.example", KEY = "<device key>", TRIP = crypto.randomUUID();
const buf = [];

navigator.geolocation.watchPosition(
  (pos) => buf.push({
    t: new Date(pos.timestamp).toISOString(),
    lat: pos.coords.latitude, lon: pos.coords.longitude,
    acc: pos.coords.accuracy, spd: pos.coords.speed, hdg: pos.coords.heading,
  }),
  console.error,
  { enableHighAccuracy: true, maximumAge: 0, timeout: 10000 }
);

setInterval(async () => {
  if (!buf.length) return;
  const fixes = buf.splice(0, 500);
  try {
    const r = await fetch(`${API}/v1/ingest/batch`, {
      method: "POST",
      headers: { "Content-Type": "application/json", "X-API-Key": KEY },
      body: JSON.stringify({ trip_id: TRIP, fixes }),
    });
    if (!r.ok) throw new Error(r.status);
  } catch { buf.unshift(...fixes); }                    // keep and retry next tick
}, 10000);
```

Listing A.2 - GPX simulator (needs ALLOW_OLD_TIMESTAMPS=true on the server)

```
# scripts/simulate_gpx.py  usage: python simulate_gpx.py drive.gpx
import sys, time, uuid, requests, gpxpy

API, KEY = "http://localhost:8000", "dev-key"
gpx = gpxpy.parse(open(sys.argv[1]))
pts = [p for tr in gpx.tracks for sg in tr.segments for p in sg.points]
trip = str(uuid.uuid4())

for i in range(0, len(pts), 10):
    fixes = [{"t": p.time.isoformat(), "lat": p.latitude, "lon": p.longitude, "acc": 5.0}
             for p in pts[i:i + 10]]
    r = requests.post(f"{API}/v1/ingest/batch", json={"trip_id": trip, "fixes": fixes},
                      headers={"X-API-Key": KEY}, timeout=5)
    r.raise_for_status()
    time.sleep(1)                       # 10 fixes per second of wall time = fast replay
```